

Document 10 — Agent Workflows → Project 3

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

[09_langgraph](#)

How to Read & Practice This Document

- **What:** putting tools, RAG, branching, and saved state together into one working agent.
- **Why:** this is what "building an agent" actually looks like in real work — combining what you already know, not new theory.
- **When:** this is the shape of most real single-agent production systems you'll run into at a job.
- **How to practice:** 1. Skim the reading list — you already know the ideas, you're just combining them now. 2. Add pieces one at a time (**Basic** → **Intermediate**): search as a plain tool first, then branching. Don't connect everything at once. 3. Do the **Real-world/Edge case** exercises with the full graph actually running, start to finish. 4. Do the **Failure** exercise on purpose — a search failing in the middle of a run shouldn't lose saved state. 5. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say "**Show me the solution.**" 6. Before moving on, explain out loud, step by step, what happens when search finds nothing useful. If you can't, you're not done.

Jump to: [Core Concepts](#) · [Practice Exercises](#) · [Build Task](#)

The Story — what this document is actually building

Picture the agent you've been building since Doc06 as a person starting their first real job. Doc06 taught it to use one tool. Doc07 taught it to keep trying and stop when it's actually done. Doc08 gave it a filing cabinet full of your documents it can search. Doc09 taught it to make decisions and remember where it left off if it gets interrupted. Every one of those was practiced alone, in its own little exercise, disconnected from the rest.

Document 10 is the day all of that has to work together at once, on a real task. A request comes in. The agent has to decide, on its own, whether this request even needs the filing cabinet — searching when you don't need to wastes time and money, and can even hand the model confusing text that pulls it off track. If it does need to search, it searches, and whatever it finds has to travel with the

task, not get lost — because a few steps later, the agent is going to pause and show a human what it found, and ask "should I go ahead?" before doing anything that can't be undone. That pause is the whole point of Doc09's saved-state trick: the agent can stop completely, wait for a real person to say yes or no — seconds, minutes, or a lunch break later — and then pick up again exactly where it left off, with nothing lost.

That's the whole story: **routing** decides if search is needed. **Search** (from Doc08) finds grounding text when it is. **State** (from Doc09) carries that found text forward so nothing has to be re-searched or re-explained. And an **interrupt** (also Doc09) makes the agent stop and ask before it acts. Put together, this is what a real, single-agent production system looks like — and it's exactly what the Build Task below, **Project 3**, asks you to build: one LangGraph agent that searches your documents when it needs to, pauses for your approval before acting, and finishes with an answer you can actually trust, because you can see exactly what it found and what it did with it.

Core Concepts (read this first — everything you need is here)

This document doesn't introduce **anything new** — the skill here is combining Docs 06-09 (tools, agents, RAG, graphs) into one working system, so what needs explaining is how the pieces fit, not new ideas.

Search as a routed step, not something that always runs

In Doc08, `retrieve()` was a stand-alone function. Here, it becomes a graph node — but importantly, not one that always runs. A conditional edge (Doc09) decides, based on the current task in the state, whether the search step should run at all. **Why this matters:** not every task needs grounding in your documents — skipping search when it's not needed saves time and money, and more importantly, it stops the model from being handed unrelated retrieved text that could pull it off track.

Why "agentic RAG" is really a graph problem, not a search problem

Plain RAG (Doc08) is a fixed pipeline: always search, always answer from what was found. "Agentic" RAG makes search a *decision* the graph makes on its own — search only if needed, maybe search *again* with a better question if the first try found nothing useful, and follow a separate "can't answer" path when there's genuinely nothing to ground the answer in. **Why this only becomes possible now, not back in Doc08:** it needs exactly the conditional-edge and state tools Doc09 just gave you — plain functions can't cleanly express "try again with different settings if this path fails" the way graph structure can.

State must carry what was found, not just what was asked

When a human-checking pause happens *after* search, the state saved at that pause point needs to include the found text itself — not just the original question — so that resuming doesn't need to

search again (which could return different results if the documents changed by then), and so the final answer stays grounded in exactly what was shown to the person who approved it. **Why this is worth pointing out clearly:** it's an easy mistake to save too little state, only to find out during a resume that you're missing something you need to continue correctly.

Multi-step search: when one search isn't enough

This document's exercises focus on a single search-then-answer flow: decide if search is needed, search once, answer using what was found. Some real questions don't fit that shape — the first search comes back with results that are close but not quite right, and reading them is the only way to know what a *better* search would look like. **When a real feature needs this:** any time the right query depends on information the agent doesn't have until after it has already searched once — for example, "what was the reason for the delay mentioned in the March report" might need one search to find the March report, then a second, more specific search for the actual reason inside it. **How it contrasts with this document's flow:** instead of a straight line from search to answer, the routing function after search checks whether the results look sufficient — if not, it sends the graph back to a "refine query" node and then back to search again, which is exactly the "loop on purpose" pattern from Doc09, with a real stopping rule (a max number of search attempts) so it can't loop forever.

Combining retrieval with tool calls

It's easy to think of "search my documents" (Doc08's RAG) and "call a tool" (Doc06's tool-calling) as two different mechanisms because they live in different documents — they aren't. A retriever is just a function that takes a query and returns some text, which is exactly what a `@tool`-decorated function is. **Why this matters:** once your document search is registered as a tool the same way you'd register a weather API or a calculator, the model can freely decide, in the same reasoning loop, to call "search my documents" *and* "check this order's status via an API" *and* "search my documents again" — all as ordinary tool calls, with no special-case code needed to treat retrieval differently from any other tool. **How it works:** wrap `retrieve()` with `@tool` exactly the way you'd wrap any other function (this document's Basic exercise already does this), register it alongside your other tools on the model-calling node, and let the model's own tool-choosing logic decide when to use which.

```
@tool
def search_documents(query: str) -> str:
    """Search the knowledge base for relevant text."""
    return retrieve(query)

model_with_tools = model.bind_tools([search_documents, check_order_status])
```

Caching search results within one run

If your graph can route back through the search node more than once — from the multi-step search pattern above, or from a loop that revisits an earlier decision — it's easy to end up re-running the *same* search, re-embedding the *same* query text and re-querying the vector store, when the answer is already sitting in state from a few steps ago. **Why this wastes both time and money:** embedding a query costs an API call, and searching a vector store takes real time — doing that twice for the same question inside one run buys you nothing, since the documents haven't changed in the seconds or minutes the graph has been running. **The basic idea:** keep a simple cache — a plain Python dictionary mapping query text to its search results — that lives for the lifetime of one graph run (it can just be a field in state). Before searching, check the cache first; only call the real retriever on a cache miss, and store the result afterward for next time.

```
def retrieve_node(state):
    query = state["query"]
    cache = state.get("search_cache", {})
    if query in cache:
        return {"found_text": cache[query]}
    result = retrieve(query)
    cache[query] = result
    return {"found_text": result, "search_cache": cache}
```

When human approval should block vs. just notify

This document's `approval_pause` exercise uses `interrupt()` the way you'd expect: the graph stops completely and waits for a person to say yes or no before doing anything else. That's the right choice for some actions, but not all of them. **Blocking approval (pause and wait):** fits actions that are hard or impossible to undo and genuinely risky if wrong — sending an email to a customer, charging a payment, deleting a record. **"Notify but proceed" (log it and keep going):** fits actions that are low-risk, easy to reverse, or where waiting on a human would slow things down for no real safety benefit — updating an internal draft, tagging a record for later review, posting to an internal Slack channel. **When to use which:** ask how reversible the action is, and how costly it is if wrong — high-risk, hard-to-reverse actions should block with `interrupt()` ; low-risk, easy-to-reverse actions can just write a log entry and carry straight on, so a human reviews afterward instead of being a bottleneck for every single step.

```
def maybe_approve(state):
    if state["action"]["risk"] == "high":
        return interrupt({"action": state["action"]})
    state["audit_log"].append(state["action"])
    return state
```

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- [LangGraph — tutorials index](#) — the "RAG agent" / "agentic RAG" style tutorials.
- Look again at [08_rag](#)'s reading list — you're combining that search tool now, not learning new search theory.

Practice Exercises

Setup for this document's practice code: work inside `10_agent_workflows/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install langgraph langchain-openai chromadb`.

How to run each exercise: group your practice code by topic, not by difficulty level. If two exercises below are really about the same thing, save them together in ONE script named after that topic — for example, if two exercises are both about `.env` config, save both in one file like `env_config_practice.py`, with each level's version as its own clearly labeled section inside it. Run each topic's file directly, for example: `python env_config_practice.py`.

For this document: - Basic (`search_as_tool`) and Intermediate (`conditional_search`) are both about wiring search into the graph — first as a plain tool, then routed conditionally — save them together as `search_tool_integration_practice.py`, with each level as its own section. - Real-world (`approval_pause`) is its own topic — save it as `approval_pause_practice.py`. - Edge cases (`empty_search`) is its own topic — save it as `empty_search_practice.py`. - Failure (`search_failure`) is its own topic — save it as `search_failure_practice.py`.

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to advanced, side by side.

Jump to an exercise: [Basic](#) · [Intermediate](#) · [Real-world](#) · [Edge cases](#) · [Failure](#) · [Build Task](#)

Basic — plug search in as a plain tool first

- **What:** add the Doc08 search tool as a plain tool your Doc09 graph can call, not yet conditionally routed.
- **Why:** connect the pieces one at a time — search-as-a-tool working, before search-as-conditionally-routed. Two problems at once is harder to debug than one.
- **When you'll hit this for real:** the very first step of wiring RAG into any agent graph.
- **How to code it:** wrap `08_rag`'s `retrieve()` as a `@tool`, register it on your Doc09 graph's model-calling node, and confirm the graph can call it when asked a question about your

document set.

- **Save as:** `search_tool_integration_practice.py`, under a `# Basic` section (this file also holds the Intermediate exercise below, in its own `# Intermediate` section).
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — make search conditional

- **What:** a conditional edge that only routes to search when the task actually needs it, skipping it otherwise — proven on both paths.
- **Why:** this is the exact mechanism that keeps your graph from wasting a search call (and its cost) on tasks that don't need grounding.
- **When you'll hit this for real:** this document's own Build Task, and any real agent handling a mix of "needs my documents" and "doesn't" questions.
- **How to code it:** write a routing function checking whether the task looks like it needs the knowledge base, wire it with `add_conditional_edges`, and run one test question that should search and one that shouldn't — confirm both paths.
- **Save as:** `search_tool_integration_practice.py`, under an `# Intermediate` section (this file also holds the Basic exercise above, in its own `# Basic` section).
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — a real approval pause

- **What:** a human-checking pause before the graph acts on found information (like "I found X, should I go ahead?").
- **Why:** this is the actual production pattern Project 3 needs — grounding an approval step in what was *found*, not just the original question.
- **When you'll hit this for real:** Project 3's own Approval-style gate, and any real agent taking a consequential action based on retrieved data.
- **How to code it:** after your reasoning node produces a draft, call `interrupt()` with the draft and its source chunks attached, then resume with a simulated "approved" input and confirm the graph continues correctly.
- **Save as:** `approval_pause_practice.py`.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Edge cases — search comes back empty

- **What:** a task where search finds nothing useful — check whether the graph carries on anyway, or follows a "can't answer" path.
- **Why:** this is the exact failure mode that turns into a hallucinated answer if you don't design for it — you need to see your own graph handle it correctly, or catch it not doing so.
- **When you'll hit this for real:** any question genuinely outside your knowledge base's coverage.

- **How to code it:** ask a question with no relevant documents in your set, and confirm your graph routes to an explicit "I don't have grounding for this" response instead of generating an ungrounded answer.
- **Save as:** `empty_search_practice.py` .
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure — search fails mid-run

- **What:** make the search call fail or time out in the middle of a run, and check the graph handles it without losing saved state.
- **Why:** an external search failure (a down vector store, a network blip) shouldn't corrupt or lose everything the graph had already done — this is a real production requirement, not a nice-to-have.
- **When you'll hit this for real:** any real deployment where your vector store is a separate service that can be briefly unavailable.
- **How to code it:** temporarily make your retrieval node raise an exception on purpose, run the graph, and confirm the checkpointed state from before the failure is still intact and inspectable afterward.
- **Save as:** `search_failure_practice.py` .
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Build Task — Project 3: LangGraph App

Stuck on the Build Task? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Goal: a LangGraph agent with state, branching, a RAG search tool, saved state, and at least one human-approval pause before a risky action — a full search → answer → (pause for approval) → resume → final answer cycle.

Requirements:

- Builds on the Doc09 graph skeleton — doesn't start from scratch.
- A search node/tool wired in from Doc08, with a conditional edge deciding when it's used.
- At least one `interrupt()` for human approval before something consequential happens (the action can still be a placeholder, but the approval step must be real).
- Shows the state's progress (at least step by step) in the terminal, not just the final answer.
- Saved state survives the interrupt/resume cycle, for a real, non-trivial task.

Inputs: a free-text task that needs search, from your Doc08 set of documents.

Outputs: a final answer grounded in the found documents; a visible log of the path taken (searched or not, approved or not).

Constraints: must correctly handle "search found nothing useful" without making up an answer anyway.

Suggested files:

```
project_3_documind_rag_agent/
├─ main.py
├─ graph.py          (builds on 09_langgraph/graph.py)
├─ state.py
├─ nodes.py
├─ retriever.py      (reused from 08_rag)
└─ test_project3.py
```

Functions/Components to build:

- `nodes.py` → a `retrieve_node` , a `reason_node` , an `approval_node`
- `graph.py` → a conditional edge function deciding when to route through search
- `main.py` → runs the graph, handles the pause/resume experience in the terminal

Expected Behavior

- Full cycle: task comes in → branching decides if search is needed → search happens if so → reasoning uses the found text → interrupt pauses before the risky action → resume continues → grounded final answer.
- A task that doesn't need search skips that path entirely and still finishes correctly.
- "Search found nothing" looks visibly different from "search succeeded" in the output/log.

Test Cases

Scenario	Expected
Task needing search, info found	Full cycle finishes, answer grounded in the found document
Task not needing search	Search path skipped, correct direct answer
Task needing search, nothing relevant found	Graph reports it can't ground the answer, doesn't make one up
Interrupt reached, approved on resume	Graph continues and finishes correctly
Interrupt reached, rejected on resume	Graph stops/reports cleanly, doesn't do the risky action

Break-It / Debug Preview

- A race between the saved-state write and a mid-run interrupt.
- A routing check that skips search on a task that actually needed it.
- Full debugging drill in [14_debugging_lab](#).

Interview Topics Preview

- How this is different from Project 2 · where saved state actually lives · what human-checking costs in delay/user experience · agentic RAG vs. plain RAG.

□ You Can Now Build Project 3

Docs 08-10 (plus the Async Python gate) are everything Project 3 needs. Go to [project_3_documind_rag_agent/](#) and start with its **Setup** section — the step-by-step build guide there (not the Build Task summary above) is what you actually follow.

Move On When

Project 3 runs a full search → answer → pause → resume → final answer cycle start to finish, in your own code. Full details: [CURRICULUM.md §4](#).

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full solution only if you say "**Show me the solution.**"